練習 16：Field-Symbol

> 後來補的主題：建議在 **ex05 之後**任何時間點教（只依賴 ex03/ex04/ex05 的 internal table 技能；答案程式用到 ex08 的 FORM，先教也不影響理解）。

學習目標

- 理解 Field-Symbol 是**別名不是複本**：`ASSIGN` 之後改 `<fs>` 等於改本尊
- 會宣告與指派：`FIELD-SYMBOLS <fs> TYPE ...`、`ASSIGN ... TO <fs>`（行內宣告 `FIELD-SYMBOL(<fs>)` 是 7.40 之後的新寫法，留到 ex26）
- 會用 `LOOP AT ... ASSIGNING` 直接修改表格內容——不用 work area、不用 `MODIFY`
- 會用 `READ TABLE ... ASSIGNING` 單筆就地修改
- 會防呆：`IS ASSIGNED`、`sy-subrc`；知道 `UNASSIGN` 後亂用會 dump
- 看得懂舊程式的 `ASSIGN COMPONENT ... OF STRUCTURE` 動態欄位存取

事前準備

建立程式 `ZR_TR16_<你的姓名縮寫>`，套件 `$TMP`。

題目需求

沿用 ex05 的學生成績資料（4 筆：85 / 92 / 67 / 45）。

1. **別名不是複本**：宣告 `gv_total TYPE i VALUE 100` 與 `FIELD-SYMBOLS <fs_num> TYPE i`；`ASSIGN` 之後把 `<fs_num>` 加 50，輸出 `gv_total`——證明改別名就是改本尊（存取前用 `IS ASSIGNED` 防呆）
2. **LOOP ASSIGNING 打等第**：先宣告 `FIELD-SYMBOLS <ls_student> TYPE ty_student`，再 `LOOP AT gt_students ASSIGNING <ls_student>.`，直接改 `<ls_student>-grade`（>=80 A、>=60 B、其餘 C）——**整段沒有 work area、沒有 MODIFY**，對照 ex05 第 4 步的三段式寫法
3. **READ TABLE ASSIGNING 補考**：先宣告 `FIELD-SYMBOLS <ls_hit> TYPE ty_student`，`WITH KEY id = 'S0004'` 拿到那一列的別名，成績 +30、等第重打成 B；記得檢查 `sy-subrc`
4. **動態欄位存取**（看懂舊程式用）：先宣告 `FIELD-SYMBOLS <fs_comp> TYPE any`（每欄型別不同，只有 `any` 都能接），`DO` 迴圈配 `ASSIGN COMPONENT sy-index OF STRUCTURE gs_student TO <fs_comp>`，逐欄輸出第一筆的每個欄位值，`sy-subrc <> 0` 時 `EXIT`；並在程式裡用註解寫出每一圈 `WRITE` 會輸出什麼
5. 實驗（看完註解掉）：`UNASSIGN <fs_num>.` 之後再 `<fs_num> = 999.`——執行期 dump `GETWA_NOT_ASSIGNED`，記住這個 dump 名字，維護時會再見到它

預期輸出（範例）

```
=== 1) 別名不是複本 ===
改 <fs_num> 之後 gv_total =        150
=== 2) LOOP ASSIGNING 打等第（沒有 MODIFY） ===
S0001 王小明         85 A
S0002 李小美         92 A
S0003 陳大文         67 B
S0004 張三豐         45 C
```

```
=== 3) READ TABLE ASSIGNING 補考後 ===
S0001 王小明          85 A
S0002 李小美          92 A
S0003 陳大文          67 B
S0004 張三豐          75 B
=== 4) ASSIGN COMPONENT 逐欄輸出第一筆 ===
  欄位          1 : S0001
  欄位          2 : 王小明
  欄位          3 :         85
  欄位          4 : A
```

對比：ex05 的 MODIFY vs ex16 的 ASSIGNING

	ex05（INTO + MODIFY）	**ex16（ASSIGNING）**
LOOP 拿到的	複本（work area）	那一列本人（別名）
改完要做什麼	`MODIFY` 寫回，忘了就丟失	什麼都不用，改了就是改了
效能	每筆複製一次（欄位多時有感）	零複製
風險	忘記 MODIFY	「順手」改到不該改的欄位

團隊實務備註

- **要修改就 ASSIGNING、唯讀可以 INTO**：這是團隊建議的預設選擇。ASSIGNING 沒有「忘了 MODIFY」的坑、也省複製；唯讀場景 INTO 的複本反而是保護（改了也不影響表格）
- `LOOP AT ... ASSIGNING` 中**不要 DELETE 當前那一列**——別名會懸空，跟 ex05 思考題 2 的 sy-tabix 是同級的經典陷阱
- 舊程式（ZDQM 系列等）常見 `ASSIGN COMPONENT` 配動態欄位名做泛用邏輯，第 4 步練的就是讀懂它的基本功
- OOP 課程 op12 的 `build` 方法就是用 `ASSIGNING` 算營收——這題學的寫法會一路用到結業

思考題

1. 第 2 步的 LOOP 如果把 `ASSIGNING <ls_student>` 換回 `INTO gs_student`，但**忘了寫 MODIFY**，輸出會變怎樣？（就是 ex05 思考題 1——現在你知道兩種解法了）
2. `READ TABLE ... ASSIGNING` 找不到資料時 `<ls_hit>` 是什麼狀態？不檢查 `sy-subrc` 直接用會發生什麼事？
3. 唯讀的 LOOP（只 WRITE 不改）用 ASSIGNING 也比較快，那為什麼團隊備註還建議唯讀用 INTO？（提示：複本是保護——手滑改到 `<ls>` 的欄位，錯誤會直接進表格）

答案

見 `zr_tr16_field_symbols.prog.abap`（SAP 端程式 `ZR_TR16_FIELD_SYMBOLS`）。

新式寫法版本見 `zr_tr16_field_symbols_new.prog.abap`（SAP 端程式 `ZR_TR16_FIELD_SYMBOLS_NEW`），輸出跟原版逐行比對相同；新舊寫法的對照說明見講義 16 §7的「新式寫法對照」。